(U) ODEDSA Interface Control Document



DRAFT

Prepared By:
Revision: 4.0
Date: 20251114

Classified By:
Derived From:
Declassify On:

Controlled by:	NGIC
Distribution/Dissemination Controls:	NGIC
POC:	Mike Lesniak, NGIC
Contact:	usarmy.charlottesville.ngic.list.modsimoffice@mail.mil

11

DRAFT

(U) FOR TITLE PAGE
(U) TIMELINE: MOST ITEMS: Ready by SRR, finalized by PDR. REVISION NUMBERS & DATES: Finalized at FDR.
(U) "Prepared By" and "Classified By" must be a company name, not an individual's name
(U) All dates are in YYYYMMDD format, both "Date" and "Declassify On" (unless the declassification is not on a specific date).
(U) If the document is classified, the Classification Block should be, "Controlled By", "POC", "Contact". If the document is CUI, the Classification Block should be, "Controlled By", "CUI Category(ies)", "Limited Dissemination", "POC", "Contact". Unclassified documents have no Classification Block.
(U) "Distribution/Dissemination Controls" line is only needed if the document contains CUI material

(U) FOR REVISION TABLE:
(U) TIMELINE: Each milestone submission needs its own row, in 0.# format. Final delivery is 1.0. Dates are YYYYMMDD.

UNCLASSIFIED

Revision	Date	Author	Description of Changes
1.0	20240908	BAH	New Template
1.1	20241226	BAH	Updates to template based on NGIC feedback
1.2	20250115	BAH	Updates to template based on NGIC feedback
3.0	20250217	BAH	Baselined to 3.0 release for document templates
4.0	20251114	BAH	Baselined to 4.0 release for document templates

UNCLASSIFIED

DRAFT

(U) Table of Contents

1 (U) Introduction 1

2 (U) Base External Simulation Interface 1

2.1 (U) Functions . . . 1

2.1.1 (U) Update . . . 2

2.1.2 (U) Cleanup . . . 2

2.1.3 (U) Destroy System . . . 3

2.1.4 (U) Get Current Timestep Start . . . 3

2.1.5 (U) Get Current Timestep End . . . 4

2.1.6 (U) Get Current Timestep Int . . . 4

2.1.7 (U) Get Simulation Timestep Size . . . 5

2.1.8 (U) Add External System . . . 5

2.1.9 (U) Add External System (Extended) . . . 6

2.1.10 (U) Update External System . . . 6

2.1.11 (U) Initialize Environment . . . 7

2.1.12 (U) Add External SPDW . . . 7

2.1.13 (U) Get VOACAP Angles . . . 8

2.1.14 (U) Get External System Info Extended . . . 9

2.2 (U) Structs . . . 10

3 (U) Sources 16

4 (U) Acronyms 17

DRAFT

(U) List of Tables

1 (U) Export Functions . . . 1
2 (U) Update . . . 2
3 (U) Cleanup . . . 3
4 (U) Destroy System . . . 3
5 (U) Get Current Timestep Start . . . 4
6 (U) Get Current Timestep End . . . 4
7 (U) Get Current Timestep Int . . . 4
8 (U) Get Current Timestep Size . . . 5
9 (U) Add External System . . . 6
10 (U) Add External System (Extended) . . . 6
11 (U) Update External System . . . 7
12 (U) Initialize Environment . . . 7
13 (U) Add External SPDW . . . 8
14 (U) Get VOACAP Angles . . . 8
15 (U) Get External System Info Extended . . . 9
16 (U) External System Information Struct . . . 10
17 (U) Add External System Information Struct (Extended) . . . 11
18 (U) External Entity Information Struct . . . 11
19 (U) External Attachment Information Struct . . . 12
20 (U) External Transmitter Information Struct . . . 12
21 (U) Update External System Information Struct . . . 12
22 (U) Platform Information Struct . . . 13
23 (U) SPDW Initialization Struct . . . 13
24 (U) SPDW Information Struct . . . 14
25 (U) VOACAP Angles Struct . . . 15
26 (U) Azimuth and Elevation Struct . . . 15
27 (U) References . . . 16

DRAFT

64

(U) List of Figures

DRAFT

1 (U) Introduction

(U) TIMELINE: This information should be filled out by PDR.
(U) If your model is not being wrapped for integration, this document can simply state that the model is only designed to interface with the ODESSA Scenario GUI and currently does not support external integration.

(U) This document describes the use of the compiled dynamic link library (DLL) of the Overarching Dynamic Electronic-Warfare System Standard Architecture (ODESSA) framework. Booz Allen Hamilton (BAH) creates models for use with ODESSA in support of the NGIC to facilitate integration of the model and ODESSAinto external simulation environments.

2 (U) Base External Simulation Interface

(U) This section is broken down into base level functions and a section for the structs which those functions utilize.

2.1 (U) Functions

TIMELINE: This table should be drafted with all anticipated functions by SRR, and should be finalized by CDR. It should include the intended functions accessed through the DLL. As development progresses, this should be updated.

(U) This section contains a list of all base interface functions in the DLL with descriptions and explanations for each of them. These functions are common across all ODESSA external simulation DLLs.

Table 1: (U) Export Functions

UNCLASSIFIED
Function
bool Update()
bool Cleanup()
void DestroySystem()
double GetCurrentTimestepStart()
double GetCurrentTimestepEnd()
int GetCurrentTimestepInt()
double GetSimTimeStepSize()
void AddExternalSystem(AddExternalSystemInfo system_info)
void AddExternalSystem(AddExternalSystemInfo_Extended system_info_extended)
UNCLASSIFIED

Continued on next page

DRAFT

Table 1: (U) Export Functions (Continued)

UNCLASSIFIED
Function
void UpdateExternalSystem(ExternalSystemUpdateInfo system_update)
void InitEnvironment()
void AddExternalSpdw(SpdwInit spdw_init)
VoacapAngles GetVoacapAngles(source_component_id, dest_component_id, frequency_hz)
ExternalSystemInfoExtended GetExternalSystemInfo_Extended()
UNCLASSIFIED

2.1.1 (U) Update

(U) The **Update** function is a call applied to all ODESSA objects within the DLL. First the *System* updates itself, then the *Environment*. **Update** takes no input parameters and does not provide ODESSA objects with any information about the rest of the simulation. The accompanying *model specific* ICD should contain a **SetInput** function that provides simulation data *to* the ODESSA object. In most cases the *model specific* **SetInput** function should be called before **Update** is. This function throws a *generic error* described below. *Note: other functions reference this throw error, therefore it is written generically.* The accompanying *model specific* ICD should contain a function which initializes the ODESSA system object in the DLL. If the **FUNCTION** is called before the *model specific* initialization function is called, **FUNCTION** will throw a *generic error*. This *generic* throw will print an error message which reads:

"Error: cannot call function **FUNCTION** before system is initialized."

Table 2: (U) Update

UNCLASSIFIED	
Update	
bool Update()	
Parameters	Description
	No input value taken
Return	Description
bool	A bool which always returns true, this provides no meaningful information
UNCLASSIFIED	

2.1.2 (U) Cleanup

(U) The **Cleanup** function is a call applied to all ODESSA objects within the DLL. First the *System* performs its end of timestep cleanup functionality, then any internal representations of external systems perform theirs. **Cleanup** takes no input parameters and does not provide ODESSA objects with any information about the rest

of the simulation. **Cleanup** should be called at the end of every simulation timestep. This function throws a *generic error* as described above.

Table 3: (U) Cleanup

UNCLASSIFIED	
Cleanup	
bool Cleanup()	
Parameters	Description
	No input value taken
Return	Description
bool	A bool which always returns true, this provides no meaningful information
	UNCLASSIFIED

2.1.3 (U) Destroy System

(U) The **DestroySystem** function is used at the very end of a simulation to release memory allocated within the DLL. This function does not throw errors.

Table 4: (U) Destroy System

UNCLASSIFIED	
DestroySystem	
void DestroySystem()	
Parameters	Description
	No input value taken
Return	Description
void	No return value given
	UNCLASSIFIED

2.1.4 (U) Get Current Timestep Start

(U) The **GetCurrentTimestepStart** returns the simulation time in seconds at the beginning of the current timestep. This function throws a *generic error* as described above.

DRAFT

Table 5: (U) Get Current Timestep Start

UNCLASSIFIED	
GetCurrentTimestepStart	
double GetCurrentTimestepStart()	
Parameters	Description
	No input value taken
Return	Description
double	The beginning of the current simulation timestep in seconds
	UNCLASSIFIED

2.1.5 (U) Get Current Timestep End

(U) The **GetCurrentTimestepEnd** returns the simulation time in seconds at the end of the current timestep. This function throws a *generic error* as described above.

Table 6: (U) Get Current Timestep End

UNCLASSIFIED	
GetCurrentTimestepEnd	
double GetCurrentTimestepEnd()	
Parameters	Description
	No input value taken
Return	Description
double	The end of the current simulation timestep in seconds
	UNCLASSIFIED

2.1.6 (U) Get Current Timestep Int

(U) The **GetCurrentTimestepInt** returns the current simulation timestep as an integer count. This function throws a *generic error* as described above.

Table 7: (U) Get Current Timestep Int

UNCLASSIFIED
GetCurrentTimestepInt
int GetCurrentTimestepInt()
UNCLASSIFIED

Continued on next page

DRAFT

Table 7: (U) Get Current Timestep Int (Continued)

UNCLASSIFIED	
GetCurrentTimestepInt	
Parameters	Description
	No input value taken
Return	Description
double	The current simulation timestep as an integer count
	UNCLASSIFIED

2.1.7 (U) Get Simulation Timestep Size

(U) The **GetCurrentTimeStepSize** function returns the length of the internal DLL timestep as a double in seconds. This value is set through a selected *.scn* file and should remain the same throughout the simulation runtime. This function throws a *generic error* as described above.

Table 8: (U) Get Current Timestep Size

UNCLASSIFIED	
GetSimTimeStepSize	
double GetSimTimeStepSize()	
Parameters	Description
	No input value taken
Return	Description
double	Returns the length of the internal DLL timestep in seconds
	UNCLASSIFIED

2.1.8 (U) Add External System

(U) The DLL has an internal representation of entities from the external simulation of which the model or the environment need to be aware. These representations are added with the **AddExternalSystem** function(s). This function applies inputs in values from the ***Add External System Info Struct*** to a new asset. This function should be called before the **InitEnvironment** function so that the ODESSA environment inside the DLL can properly register these representations. This function throws a *generic error* as described above.

DRAFT

Table 9: (U) Add External System

UNCLASSIFIED	
AddExternalSystem	
void AddExternalSystem(AddExternalSystemInfo system_info)	
Parameters	Description
AddExternalSystemInfo system_info	A ***struct*** containing relevant system info values
Return	Description
void	No return value given
	UNCLASSIFIED

2.1.9 (U) Add External System (Extended)

(U) The DLL has an internal representation of entities from the external simulation of which the model or the environment need to be aware. These representations are added with the **AddExternalSystem** function(s). This function applies inputs in values from the ***Add External System Info Struct (Extended)*** to a new asset. This function should be called before the **InitEnvironment** function so that the ODESSA environment inside the DLL can properly register these representations. This function throws a *generic error* as described above.

Table 10: (U) Add External System (Extended)

UNCLASSIFIED	
AddExternalSystemExtended	
void AddExternalSystem(AddExternalSystemInfo_Extended system_info_extended)	
Parameters	Description
AddExternalSystemInfo_Extended system_info	A ***struct*** containing relevant system info values
Return	Description
void	No return value given
	UNCLASSIFIED

2.1.10 (U) Update External System

(U) The **UpdateExternalSystem** function updates the positional data of an entity added to the simulation through the DLL with the **AddExternalSystem** function(s). The external system is specified with the *system_id* member variable in the ***Update External System Info Struct***. This function throws a *generic error* as described above.

DRAFT

Table 11: (U) Update External System

UNCLASSIFIED	
UpdateExternalSystem	
void UpdateExternalSystem(ExternalSystemUpdateInfo system_update)	
Parameters	Description
system_update	A ***struct*** containing relevant external system values
Return	Description
void	No return value given
	UNCLASSIFIED

2.1.11 (U) Initialize Environment

(U) The **InitEnvironment** function creates and initializes the ODESSA environment within the DLL using input from a selected *.scn* file. The ODESSA environment is responsible for handling emitted Simulated Pulse Descriptor Word (SPDW)s from external systems as they travel through the atmosphere. In order for SPDWs to be received by the model this **InitEnvironment** function must be called before the **Update** function. **AddExternalSystem** calls should be performed before this function. This function throws a *generic error* as described above.

Table 12: (U) Initialize Environment

UNCLASSIFIED	
InitEnvironment	
void InitEnvironment()	
Parameters	Description
	No input value taken
Return	Description
void	No return value given
	UNCLASSIFIED

2.1.12 (U) Add External SPDW

(U) The **AddExternalSpdw** injects an SPDW into the outbox of a generic transmitting component that represents the transmitter of an external system. This function should be called when a receiver needs to accept pulses from other models in the external simulation. In order to call it an external model must be added as an external system and the **InitEnvironment** function must have been called; this will allow external models to appear in propagation paths. This function throws a *generic error* as described above.

DRAFT

Table 13: (U) Add External SPDW

UNCLASSIFIED	
AddExternalSpdw	
void AddExternalSpdw(SpdwInit spdw_init)	
Parameters	Description
spdw_init	A ***struct*** containing relevant SPDW values
Return	Description
void	No return value given
	UNCLASSIFIED

2.1.13 (U) Get VOACAP Angles

(U) The **GetVoacapAngles** collects takeoff and landing angles calculated by Voice of America Coverage Analysis Program (VOACAP) for skywave propagation. See the UAM for additional information on VOACAP. This function throws a *generic error* as described above.

(U) ODESSA system objects in the DLL will also throw an error if the VOACAP input frequency is outside the operation limits. These limits are within two megahertz and thirty megahertz. This throw will print an error message which reads:

"Error: VOACAP input frequency must be between 2 MHz and 30 MHz."

Table 14: (U) Get VOACAP Angles

UNCLASSIFIED	
GetVoacapAngles	
VoacapAngles GetVoacapAngles(source_component_id, dest_component_id, frequency_hz)	
Parameters	Description
source_component_id	Integer which describes the component which the signal emits from
dest_compnent_id	Integer which describes the component which the signal is to be received by
frequency_hz	Double which describes center frequency of an associated signal being propagated (hertz)
Return	Description
VoacapAngles	A ***struct*** which contains the azimuthal and elevation angles for the takeoff and landing for skywave propagation
	UNCLASSIFIED

DRAFT

2.1.14 (U) Get External System Info Extended

(U) The function **GetExternalSystemInfo_Extended** returns a struct representation of a model's system that can be given to other systems in the simulation. Other systems can use this struct to inform their copy of the environment code that a new system has entered the simulation. The environment can interact with this shell model system without having to know about the real model which lives in a different dll.

Table 15: (U) Get External System Info Extended

UNCLASSIFIED	
GetExternalSystemInfoExtended	
ExternalSystemInfoExtended GetExternalSystemInfo_Extended()	
Parameters	Description
	No input value taken
Return	Description
ExternalSystemInfoExtended	A struct which contains the info needed to create a shell aproximation of this model in a different dll
	UNCLASSIFIED

DRAFT

2.2 (U) Structs

(U) This section contains the structs that are common across all ODESSA external simulation DLLs.

Table 16: (U) External System Information Struct

UNCLASSIFIED	
addTarget	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
int system_id	Integer ID of the external system
int entity_id	Integer ID of the entity within the external system
int platform_id	Integer ID of the platform within the external system
bool has_tx	Bool value describing whether the external system has a transmitter or not
int interface_id	Integer ID of the interface within the external system. Only used if 'has_tx' is true
int tx_id	Integer ID of the transmitter within the external system. Only used if 'has_tx' is true
int number_of_hits_to_death	Integer containing value of "hits to kill" an asset
double rcs	Double containing the radar cross section, the apparent size of an asset in meters squared
double initial_speed	Double containing initial speed in meters per second of an asset
double initial_heading	Double containing initial directional facing of an asset in radians where 0.0 is due north
std::array<double, 3> dimensions	An array containing the dimensions of a target in length(m), width(m), and height(m) respectively
std::array<double, 3> initial_lla	Initial LLA of a target in latitude(radians), longitude(radians), and altitude(m) respectively
double operating_frequency_hz	Double containing the operating frequency in Hertz of the external system's transmitter. Only used if 'has_tx' is true
	UNCLASSIFIED

DRAFT

Table 17: (U) Add External System Information Struct (Extended)

UNCLASSIFIED	
addExternalSystemExtended	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
int system_id	Integer ID of the external system
Color alignment	Alignment of the external system (red, blue, or gray)
std::vector<ExternalEntityInfo> entity_info	Vector of external entity information structs to represent entities within the external system
	UNCLASSIFIED

Table 18: (U) External Entity Information Struct

UNCLASSIFIED	
externalEntityInfo	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
int entity_id	Integer ID of the entity within the external system.
int platform_id	Integer ID of the platform within the external system.
std::vector<ExternalAttachmentInfo> attachment_info	Vector of external attachment info structs to represent the platform attachments of the external entity.
int number_of_hits_to_death	Integer containing value of "hits to kill" an asset.
double rcs	Double containing the radar cross section, the apparent size of an asset, in meters squared.
double initial_speed	Double containing initial speed in meters per second of an asset.
double initial_heading	Double containing initial directional facing of an asset in radians where 0.0 is due north.
std::array<double, 3> dimensions	An array containing the dimensions of a target in length(m), width(m), and height(m) respectively.
std::array<double, 3> initial_lla	Initial LLA of a target in latitude(radians), longitude(radians), and altitude(m) respectively.
	UNCLASSIFIED

DRAFT

Table 19: (U) External Attachment Information Struct

UNCLASSIFIED	
externalAttachmentInfo	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
int interface_id	Integer ID of the external interface for this grouping of attachments
std::vector<ExternalTransmitterInfo> transmitter_info	Vector of external transmitter information structs to represent transmitters within this grouping of attachments
	UNCLASSIFIED

Table 20: (U) External Transmitter Information Struct

UNCLASSIFIED	
externalTransmitterInfo	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
int transmitter_id	Integer ID of the transmitter
double operating_frequency_hz	Double containing the operating frequency in Hertz of the transmitter
	UNCLASSIFIED

Table 21: (U) Update External System Information Struct

UNCLASSIFIED	
ExternalSystemUpdateInfo	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
int system_id	Integer ID of the external system
double update_time	Simulation time in seconds for this update information
std::vector<PlatformInfo> new_ecf_states	Updated target position and heading for each entity within the external system. These values are a described in the ***Platform Information Struct***
	UNCLASSIFIED

DRAFT

Table 22: (U) Platform Information Struct

UNCLASSIFIED	
PlatformInfo	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
std::array<double, 3> ecf_position	Earth-centered, Earth-fixed coordinates of the platform. Described as an array of doubles: X (meters), Y (meters), and Z (meters)
std::array<double, 3> ecf_velocity	Earth-centered, Earth-fixed velocity of the platform. Described as an array of doubles: X (meters per second), Y (meters per second), and Z (meters per second)
std::array<double, 3> ecf_acceleration	Earth-centered, Earth-fixed acceleration of the platform. Described as an array of doubles: X (meters per second squared), Y (meters per second squared), and Z (meters per second squared)
std::array<double, 4> pointing_quaternion	Current pointing quaternion of the platform. Described as an array of doubles: q1 (unitless), q2 (unitless), q3 (unitless), and q4 (unitless)
	UNCLASSIFIED

Table 23: (U) SPDW Initialization Struct

UNCLASSIFIED	
SpdwInit	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
int external_system_id	ID of the external system that generated the associated SPDW
int sender_id	ID of the component within the external system that generated the associated SPDW
int signal_link	An integer used to describe a class/group of pulses meant to be received as a burst
int pulse_number	An integer used to describe the number of a pulse in a 'signal_link' group of pulses
std::string metadata	A string of key-value pairs describing model-specific metadata
SPDWInfo SPDW_data	Qualities relevant to the specified SPDW, these values are a described in the ***SPDW Information Struct***
	UNCLASSIFIED

DRAFT

Table 24: (U) SPDW Information Struct

UNCLASSIFIED	
SpdwInfo	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
double transmit_time_secs	Double which describes the transmit time of an signal (seconds)
double pri_secs	Double which describes the Pulse Repetition Interval of an associated signal (seconds)
double pulse_width_secs	Double which describes pulse width of an associated signal (seconds)
double erp_watts	Double which describes effective radiate power of an associated signal. This value is calculated post-environmental effects (watts)
double center_freq_hertz	Double which describes operational center frequency of an associated signal (hertz)
double signal_bandwidth_hertz	Double which describes the operational bandwidth of an associated signal (hertz)
double transmit_phase_rads	Double which describes the phase of an output antenna skin of an associated signal (radians)
CoherencyType coherency	Enum which describes the coherency of an associated signal. Includes coherent and noncoherent
SkirtShape skirt_shape	Enum which describes the skirt shape of an associated signal
JammingTypes jamming_type	Enum which describes the jamming type of an associated signal *Note: for jamming SPDWs only*
Polarization polarization	Enum which describes the polarization of the transmitting antenna of an associated signal
PulseModulation modulation_on_pulse	Enum which describes the modulation schema on the pulse of an associated signal
PulseModulationSpec modulation_spec	Enum which describes advanced parameters of the modulation of an associated signal
TransmitMode transmit_mode	Enum which describes the transmit mode of an associated signal
	UNCLASSIFIED

DRAFT

Table 25: (U) VOACAP Angles Struct

UNCLASSIFIED	
VoacapAngles	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
AzimuthElevation landing_rads	Landing angle from source component to destination component as calculated by VOACAP (radians) It is composed of by the ***Azimuth and Elevation Struct***
AzimuthElevation takeoff_rads	Takeoff angle from source component to destination component as calculated by VOACAP (radians) It is composed of by the ***Azimuth and Elevation Struct***
	UNCLASSIFIED

Table 26: (U) Azimuth and Elevation Struct

UNCLASSIFIED	
AzimuthElevation	
Functions Using This Struct	
FUNCTION NAMES	
Member	**Description**
double azimuth	Double describing the azimuth angle (radians)
double elevation	Double describing the elevation angle (radians)
	UNCLASSIFIED

DRAFT

3 (U) Sources

(U) **Table 27** lists the sources that were consulted to develop this Interface Control Document (ICD) document.

(U) TIMELINE: Should be drafted by SRR and finalized by PDR unless later intel is expected and agreed to be included after PDR by both parties.
(U) This table should reference only those source documents used to write this document, such as style guides, clock standards, code standards, etc. Include corresponding inline citations in the paragraphs where information from each source is used. Further guidance on citation style and format to come in a later update.

Table 27: (U) References

UNCLASSIFIED			
Number	Title	Date	Declass Date
			UNCLASSIFIED

DRAFT

4 (U) Acronyms

UNCLASSIFIED

Term	Description	Page
BAH	*Booz Allen Hamilton*	1
DLL	*dynamic link library*	1, 2, 3, 5, 6, 7, 8, 10
ICD	*Interface Control Document*	16
ID	*Identity*	10, 11, 12, 13
LLA	*Latitude, Longitude, and Altitude*	10, 11
NGIC	*National Ground Intelligence Center*	iii, 1
ODESSA	*Overarching Dynamic Electronic-Warfare System Standard Architecture*	1, 2, 5, 6, 7, 8, 10
SPDW	*Simulated Pulse Descriptor Word*	7, 8, 13, 14
VOACAP	*Voice of America Coverage Analysis Program*	8, 15

UNCLASSIFIED

DRAFT

DRAFT